7주차 RAG 실습 안내서

강의 운영 안내

인공지능융합활용은 이론과 코딩 실습을 함께 수행하는 교과목입니다. 6주차에는 임베딩과 코사인 유사도로 질문을 카테고리별로 분류했습니다. 7주차에는 PDF 문서에서 근거를 검색하고 질문에 답하는 RAG 질의응답 시스템을 구현합니다.
수업은 1교시 핵심 이론 50분, 2교시 워크북 1 실습 55분, 3교시 워크북 2 실습 55분으로 구성됩니다. 학생은 질문, 검색 근거, 페이지 번호, 유사도 점수, 응답 내용을 기록표에 정리합니다.

Colab 실습 환경

모든 실습은 Google Colab에서 Python으로 실행합니다. 실습 데이터인 PDF와 CSV 파일은 강의자가 Google Drive에 미리 올려 두었으며, 노트북의 첫 셀이 파일 ID로 자동 다운로드합니다. 다운로드가 실패하면 학생이 파일을 직접 업로드하는 방법을 사용합니다. 워크북 1은 API 키 없이 실습합니다. 워크북 2는 수업용 OpenAI API 키가 있는 경우에만 생성형 답변을 사용합니다.

RAG의 구조

RAG는 Retrieval Augmented Generation의 약어로, 우리말로는 검색 증강 생성이라고 합니다. 대형 언어 모델은 학습 시점 이후의 정보나 기관 내부 문서를 알지 못하기 때문에, 질문과 관련된 문서를 먼저 검색해서 프롬프트에 함께 넣어 주는 방식입니다.
RAG 파이프라인은 PDF 로딩, 텍스트 추출, 청킹, 벡터화, 관련 청크 검색, 프롬프트 구성, LLM 답변 생성의 순서로 실행됩니다. 앞의 다섯 단계를 검색 단계(Retrieval)라 하고, 뒤의 두 단계를 생성 단계(Generation)라고 합니다.

6주차와의 차이

6주차에는 미리 만든 예시 문장 지식베이스에서 가장 비슷한 문장을 찾아 질문의 카테고리를 분류했습니다. 7주차에는 실제 PDF 문서를 검색 대상으로 사용하고, 찾은 근거를 바탕으로 답변 문장을 만듭니다. 분류는 정해진 라벨 중 하나를 고르는 작업이고, 질의응답은 근거 문장을 이용해 새로운 답을 작성하는 작업입니다.

PDF 로딩과 텍스트 추출

pypdf 라이브러리의 PdfReader 클래스는 PDF 파일을 열어 페이지 목록을 제공합니다. 각 페이지 객체의 extract_text 메서드를 호출하면 그 페이지에 들어 있는 글자를 문자열로 돌려줍니다.
텍스트를 추출할 때는 페이지 번호를 함께 저장해야 합니다. 페이지 번호는 나중에 답변의 출처를 표시하는 메타데이터로 쓰입니다. enumerate 함수에 start=1을 지정하면 사람이 읽는 방식대로 1페이지부터 번호를 붙일 수 있습니다.
스캔한 이미지로만 이루어진 PDF는 글자 정보가 없어서 extract_text가 빈 문자열을 돌려줍니다. 이런 문서는 광학 문자 인식, 즉 OCR을 먼저 수행해야 하며 이번 실습 범위에는 포함하지 않습니다.

청킹 전략

청킹은 긴 문서를 검색하기 좋은 작은 구간으로 나누는 작업입니다. 문서 전체를 하나의 벡터로 만들면 여러 주제가 섞여서 질문과 정확히 맞는 부분을 찾기 어렵습니다. 반대로 구간이 너무 짧으면 문장의 앞뒤 맥락이 잘려서 근거로 쓰기 어렵습니다.

청크 크기(size)는 한 구간에 넣을 단어 수이고, 겹침(overlap)은 이웃한 두 청크가 공유하는 단어 수입니다. 겹침을 두면 문장이 경계에서 잘리더라도 다음 청크에 그 내용이 다시 포함됩니다.

일반적으로 겹침은 청크 크기의 10퍼센트에서 30퍼센트 정도로 설정합니다. 겹침이 청크 크기보다 크거나 같으면 다음 청크로 이동할 수 없으므로 오류로 처리해야 합니다.

이번 실습에서는 한국어 문장을 공백 기준으로 나눈 어절을 단어로 사용합니다. 기본값은 size=60, overlap=15이며, size를 30으로 줄였을 때 검색 결과가 어떻게 달라지는지 비교합니다.

TF-IDF와 코사인 유사도

TF-IDF는 어떤 용어가 한 문서 안에서 자주 나오면 점수를 높이고, 모든 문서에 흔하게 나오면 점수를 낮추는 가중치 방식입니다. TF는 단어 빈도, IDF는 역문서 빈도를 뜻합니다. scikit-learn의 TfidfVectorizer가 이 계산을 수행합니다.

한국어는 조사와 어미가 붙어 어절의 형태가 자주 바뀌므로, 이번 실습에서는 단어 대신 글자 조각인 문자 n-gram을 사용합니다. analyzer 값을 char_wb로 지정하면 어절 경계 안에서 두 글자에서 세 글자 길이의 조각을 만듭니다. fit_transform은 청크로 용어 사전을 만들고 벡터로 바꾸며, transform은 같은 사전으로 질문을 벡터로 바꿉니다.

코사인 유사도는 두 벡터 사이 각도의 코사인 값으로, 1에 가까울수록 방향이 비슷합니다. TF-IDF는 같은 글자가 겹치는지를 보는 방식이어서, 6주차의 의미 임베딩처럼 뜻이 비슷한 다른 표현을 찾아내지는 못합니다.

검색 임계값과 근거 없는 질문

검색 결과의 최고 점수가 미리 정한 임계값보다 낮으면 문서에서 근거를 찾지 못한 것으로 판단하고 "문서에서 확인할 수 없습니다"라고 응답합니다. 임계값은 정답이 있는 질문과 문서에 없는 질문의 점수 분포를 함께 보고 그 사이 값으로 정합니다.

문자 n-gram 방식은 질문에 문서와 같은 글자가 일부만 들어 있어도 점수가 생깁니다. 따라서 임계값만으로 모든 엉뚱한 질문을 걸러낼 수는 없습니다. 생성 단계에서 LLM에게 근거에 없는 내용은 모른다고 답하라는 지시를 함께 주어 한 번 더 막습니다.

프롬프트 구성과 환각 방지

환각은 언어 모델이 사실이 아닌 내용을 그럴듯하게 만들어 내는 현상입니다. RAG에서는 검색한 청크를 번호와 페이지와 함께 프롬프트에 넣고, 제공된 근거만 사용하라는 지시를 시스템 지시문에 적어 환각을 줄입니다.

답변에는 근거로 사용한 문서명과 페이지 번호를 함께 표시합니다. 사용자는 표시된 페이지를 직접 열어 답변이 맞는지 확인할 수 있습니다. 모델 응답은 temperature 값을 낮게 설정해 같은 질문에 비슷한 답이 나오도록 합니다.

API 키와 정보 보호

Colab 코드 셀에 API 키를 직접 작성하거나 GitHub에 공개하지 않습니다. 공개 저장소에 올라간 OpenAI 키는 자동으로 폐기될 수 있고, 다른 사람이 사용해 요금이 청구될 위험이 있습니다. 키는 Colab Secrets에 OPENAI_API_KEY라는 이름으로 저장하거나, getpass 입력창에 붙여넣어 사용합니다. getpass로 입력한 값은 화면과 노트북 파일에 남지 않습니다. 문서 데이터에는 개인 정보를 넣지 않습니다.

Gradio 웹 화면

Gradio는 Python 함수에 웹 입력창과 출력창을 연결해 주는 라이브러리입니다. gr.Interface에 실행할 함수, 입력 컴포넌트 목록, 출력 컴포넌트 목록을 지정하고 launch 메서드로 실행합니다. Textbox는 문장 입력과 출력, Checkbox는 참과 거짓 선택, Slider는 숫자 범위 선택에 사용합니다. Colab에서는 실행한 셀 아래에 웹 화면이 바로 표시됩니다.

학습 과제와 제출

워크북 1에서는 CSV 평가 질문으로 검색 결과를 검증하고, 정답 페이지가 1순위에 나온 비율인 Top-1 정확도와 3순위 안에 나온 비율인 Top-3 정확도를 기록합니다. 청크 크기를 바꿨을 때 정확도와 점수가 어떻게 달라지는지 비교해 설명합니다.

워크북 2에서는 Gradio 질의응답 화면을 실행하고, 문서에 있는 질문 두 개와 문서에 없는 질문 한 개의 응답과 근거를 기록합니다. 제출물은 실행 화면 캡처와 결과 기록표입니다.